Fiche d'explication — AES.java

Oct 20, 2026 · @Jonathan

1. Vue d'ensemble : comment AES.java est construit

AES.java chiffre un texte par blocs de 128 bits, en 10, 12 ou 14 rondes selon la clé (128, 192 ou 256 bits) ; tout est stocké sous forme de tableaux de bits (`int[]` de 0 et de 1).

Les choix de représentation (à pouvoir justifier à l'oral) :

- Un **octet** = 8 bits consécutifs, bit de poids fort en premier. L'octet numéro `i` d'un tableau occupe les cases `8i` à `8i+7`.
- Un **bloc / état** = 128 bits = 16 octets `s0..s15`. L'état 4x4 se remplit **par colonnes** : l'octet `i` est à la colonne `i/4` et à la ligne `i%4`.
- `keys` = les clés de ronde K0..Knr **mises bout à bout** (chaque clé fait 128 bits, quelle que soit la taille de la clé maître).
- Seuls le constructeur, `crypte` et `decrypte` sont publics ; tout le reste est `private`.

Déroulement du chiffrement d'un bloc :

1. `addRoundKey` avec K0.
2. Rondes 1 à Nr-1 : `subBytes`, `shiftRows`, `mixColumns`, `addRoundKey`.
3. Dernière ronde Nr : `subBytes`, `shiftRows`, `addRoundKey` (**pas de mixColumns**).

Clé	Nk (mots de 32 bits)	Rondes Nr	Clés de ronde	Mots générés
AES-128	4	10	11	44
AES-192	6	12	13	52
AES-256	8	14	15	60

Le déchiffrement refait les mêmes étapes **inverses, dans l'ordre inverse**, avec les clés de Knr vers K0.

2. Outils et conversions texte / bits

Ces quatre fonctions font le pont entre le texte, les octets et les bits ; toutes les autres s'appuient sur `lireOctet` et `ecrireOctet`.

Fonction	Ce qu'elle fait	Comment
`lireOctet(bits, i)`	Renvoie la valeur (0 à 255) de l'octet numéro `i`	Parcourt les 8 bits `8i..8i+7` ; à chaque bit : `valeur = (valeur << 1) | bit`
`ecrireOctet(bits, i, valeur)`	Écrit une valeur 0..255 dans l'octet numéro `i`	Pour chaque bit `b` de 0 à 7 : `(valeur >> (7-b)) & 1`
`stringToBits(message)`	Texte -> tableau de bits	Prend les octets UTF-8 du texte (`getBytes`) puis `ecrireOctet` pour chacun. UTF-8 permet les accents et la ponctuation
`bitsToString(bits)`	Bits -> texte	Regroupe par 8 bits (`lireOctet`), reforme les octets, `new String(octets, UTF_8)`

Exemple : la lettre `A` vaut 65 = `01000001` . `stringToBits("A")` donne `{0,1,0,0,0,0,0,1}` .

Pourquoi `<<` et `>>` ? Décaler à gauche multiplie par 2 (on fait de la place pour le bit suivant) ; décaler à droite puis `& 1` isole un bit précis.

3. Les clés : constructeur et Key Expansion

La clé maître aléatoire est étendue en nbRonde+1 clés de 128 bits ; c'est `createKeys` qui fait ce travail.

Fonction	Ce qu'elle fait	Comment
`AES(tailleCle)`	Constructeur	Refuse toute taille autre que 128/192/256 (`IllegalArgumentException`) ; fixe `nbRonde` à 10, 12 ou 14 ; génère la clé maître puis les clés de ronde
`genereMasterKey()`	Clé maître aléatoire	Remplit un tableau de `tailleCleMaitre` bits avec `SecureRandom.nextInt(2)` (générateur adapté à la cryptographie)
`createKeys()`	Key Expansion	Voir ci-dessous
`rotWord(mot)`	Rotation d'un mot	(a,b,c,d) devient (b,c,d,a)

Fonction	Ce qu'elle fait	Comment
`subWord(mot)`	S-box sur un mot	Remplace chacun des 4 octets par `sBox[ligne][colonne]`
`cleDeRonde(r)`	Extrait la clé Kr	Copie les bits `128r` à `128r+127` de `keys`

`createKeys()` pas à pas (mots de 32 bits = 4 octets) :

1. `nk = tailleCleMaitre / 32` (4, 6 ou 8) et `nbMots = 4 × (nbRonde+1)` (44, 52 ou 60).
2. Les `nk` premiers mots `w[0..nk-1]` sont **la clé maître elle-même**.
3. Pour chaque mot suivant `w[i]` : on part de `temp = w[i-1]`.
4. Si `i` est multiple de `nk` : `temp = subWord(rotWord(temp))`, puis on fait XOR du premier octet avec la constante `rcon[i/nk - 1]`.
5. Sinon, **seulement pour AES-256** (`nk > 6`) et si `i % nk == 4` : `temp = subWord(temp)`.
6. `w[i] = w[i-nk] XOR temp`.
7. On remet tous les mots en bits dans `keys`.

À retenir : une clé de 192 ou 256 bits ne donne pas des clés de ronde de 192 ou 256 bits ; les clés de ronde font toujours 128 bits, comme l'état. Les `rcon` (1, 2, 4, 8, 16, 32, 64, 128, 27, 54) sont des puissances de 2 dans GF(2^8).

4. Bourrage et découpage en blocs

AES ne chiffre que des blocs complets de 16 octets ; le bourrage complète le message avant le découpage.

Fonction	Ce qu'elle fait	Comment
`bourrage(bits)`	Complète à un multiple de 16 octets (PKCS7)	`n = 16 - (taille % 16)` ; ajoute `n` octets valant tous `n`
`debourrage(bits)`	Retire le bourrage	Lit le dernier octet `n`, vérifie `1 ⩽ n ⩽ 16` et que les `n` derniers octets valent `n`, sinon exception ; puis coupe `n` octets
`decoupageEnBlocs(bits)`	Découpe en blocs de 128 bits	Un tableau `int[nbBlocs][128]`, chaque bloc est une copie de 128 bits

Fonction	Ce qu'elle fait	Comment
`recollageDesBlocs(blocs)`	Opération inverse	Recopie les blocs bout à bout dans un seul tableau

Exemples : un message de 13 octets reçoit 3 octets `03 03 03`. Un message de 16 octets reçoit un bloc entier de 16 octets `10` ; c'est voulu, pour que le débourrage soit toujours possible sans ambiguïté.

Si la clé est mauvaise : le dernier octet déchiffré est quelconque, `debourrage` lève une exception, c'est ce qui permet de détecter l'erreur.

5. Les opérations d'un tour

Chaque opération prend un état de 128 bits et en renvoie un nouveau ; `confusion` vient de subBytes, `diffusion` de shiftRows + mixColumns.

Fonction	Rôle	Comment
`xor(t1, t2)`	XOR bit à bit	`t1[i] ^ t2[i]` ; renvoie un tableau vide si les tailles diffèrent
`addRoundKey(état, clé)`	Mélange la clé de ronde	Appelle `xor`. Elle est **sa propre inverse** : A xor K xor K = A. Exemple : 53 xor CA = 99
`subBytes(état)`	Non-linéarité	Chaque octet `v` devient `sBox[v >> 4][v & 15]` (4 bits forts = ligne, 4 bits faibles = colonne). Exemple : 0x53 -> ligne 5, colonne 3 -> 0xED
`subByteInv(état)`	Inverse de subBytes	Même chose avec `sBoxInv`
`shiftRows(état)`	Permutation	La ligne `l` est décalée de `l` positions vers la gauche : l'octet (colonne `c`, ligne `l`) reçoit celui de la colonne `(c+l)%4`
`shiftRowsInv(état)`	Inverse de shiftRows	Même formule dans l'autre sens (décalage à droite)
`mixColumns(état)`	Diffusion	Pour chacune des 4 colonnes, lit ses 4 octets, appelle `prodAu`, réécrit le résultat
`mixColumnsInv(état)`	Inverse de mixColumns	Pareil avec `prodAuInv`

ShiftRows sur l'exemple 00..0f : la ligne 0 ne bouge pas, la ligne 1 est décalée d'un cran, la ligne 2 de deux, la ligne 3 de trois, ce qui donne `00 05 0a 0f 04 09 0e 03 08 0d 02 07 0c 01 06 0b`.

Pourquoi des inverses ? Pour déchiffrer, il faut pouvoir annuler chaque étape : la S-box est une bijection, ShiftRows une permutation, MixColumns une matrice inversible dans GF(2^8) et AddRoundKey est involutive.

6. Les maths : multiplications dans GF(2^8)

Dans AES, les octets sont des éléments du corps GF(2^8) : l'addition est un XOR et la multiplication se ramène à la multiplication par 2 (`deux_a`) et au XOR.

Multiplication par 2 (`deux_a` , aussi appelée xtime) : on décale l'octet d'un bit vers la gauche (`<< 1` , puis `& 0xFF` pour rester sur 8 bits). Si le bit de poids fort valait 1, un bit a débordé et on fait un XOR avec `0x1B` (le polynôme irréductible x^8+x^4+x^3+x+1

réduit).

Fonction	Calcul	Décomposition
`deux_a(a)`	2a	décalage + XOR 0x1B si débordement
`trois_a(a)`	3a	2a xor a
`neuf_a(a)`	9a	8a xor a (8a = trois fois `deux_a`)
`onze_a(a)`	11a	8a xor 2a xor a
`treize_a(a)`	13a	8a xor 4a xor a
`quatorze_a(a)`	14a	8a xor 4a xor 2a

La décomposition vient de l'écriture binaire : 9 = 8+1, 11 = 8+2+1, 13 = 8+4+1, 14 = 8+4+2, et "+" devient XOR.

`prodAu(a)` multiplie une colonne `(a0,a1,a2,a3)` par la matrice de MixColumns :

```
\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03
\\ 03 & 01 & 01 & 02 \end{pmatrix}
```

Ainsi `b0 = 2·a0 xor 3·a1 xor a2 xor a3`, et ainsi de suite pour b1, b2, b3 (multiplier par 1 ne change rien).

`prodAuInv(a)` fait la même chose avec la matrice inverse, dont les lignes sont (14, 11, 13, 9), (9, 14, 11, 13), (13, 9, 14, 11) et (11, 13, 9, 14) ; c'est pour elle qu'on a besoin de `neuf_a` à `quatorze_a`.

7. Chiffrement et déchiffrement

`crypte` et `decrypte` sont les deux seules méthodes publiques (avec le constructeur) : elles enchaînent tout ce qui précède.

Fonction	Ce qu'elle fait
`crypteBloc(bloc)`	Chiffre 128 bits : addRoundKey(K0) ; pour r = 1 à Nr-1 : subBytes, shiftRows, mixColumns, addRoundKey(Kr) ; dernière ronde : subBytes, shiftRows, addRoundKey(Knr)

Fonction	Ce qu'elle fait
`decrypteBloc(bloc)`	Déchiffre 128 bits : addRoundKey(Knr), shiftRowsInv, subByteInv ; puis pour r = Nr-1 à 1 : addRoundKey(Kr), mixColumnsInv, shiftRowsInv, subByteInv ; enfin addRoundKey(K0)
`crypte(message)`	texte -> `stringToBits` -> `bourrage` -> `decoupageEnBlocs` -> `crypteBloc` sur chaque bloc -> `recollageDesBlocs`
`decrypte(bits)`	Vérifie que la taille est un multiple de 128 -> `decoupageEnBlocs` -> `decrypteBloc` sur chaque bloc -> `recollageDesBlocs` -> `debourrage` -> `bitsToString`

Pourquoi l'ordre est différent au déchiffrement ? On annule les étapes en sens inverse. À la ronde r, on avait fait SubBytes, ShiftRows, MixColumns, AddRoundKey(Kr) ; on annule donc AddRoundKey(Kr), puis MixColumns, puis ShiftRows, puis SubBytes.

Vérification faite : le chiffré obtenu est identique à celui de l'AES officiel du JDK (mode ECB, bourrage PKCS5) en 128, 192 et 256 bits, et la clé de ronde K10 correspond à l'exemple de la norme FIPS-197.

Limite à connaître : ce mode traite chaque bloc indépendamment (ECB) : deux blocs identiques donnent le même chiffré. C'est le mode demandé par le TP, mais en pratique on utiliserait CBC ou GCM.

8. Questions possibles à l'oral

- **Pourquoi des tableaux de bits ?** Pour suivre l'énoncé et rester lisible ; l'inconvénient est la lenteur par rapport à des octets.
- **Pourquoi pas de MixColumns au dernier tour ?** Il n'ajouterait pas de sécurité (il est inversible sans la clé) et complique inutilement le déchiffrement.
- **À quoi sert la Key Expansion ?** Produire des clés différentes à chaque ronde à partir d'une seule clé maître.
- **Pourquoi 11, 13 ou 15 clés de ronde ?** Une par ronde, plus K0 utilisée avant la première ronde.
- **D'où vient la confusion et la diffusion ?** La confusion vient de SubBytes (non linéaire), la diffusion de ShiftRows + MixColumns.
- **Pourquoi AddRoundKey n'a pas besoin d'inverse ?** XOR deux fois avec la même clé redonne la valeur d'origine.

- **Que se passe-t-il avec une mauvaise clé ?** Le résultat est du bruit, le bourrage est presque toujours invalide et `decrypte` lève une exception.
- **Pourquoi UTF-8 ?** Pour gérer accents et ponctuation avec la même logique octet par octet.
- **Pourquoi `SecureRandom` ?** C'est un générateur aléatoire de qualité cryptographique, contrairement à `Random`.
- **Comment avez-vous testé ?** `TestAES` et une comparaison avec `javax.crypto` (AES officiel du JDK) sur des clés et messages aléatoires.